GENERATIVE AI & AGENTIC AI · LESSON 12 OF 12

Building and Shipping a GenAI App: Architecture, Streamlit/FastAPI, Cost, Observability and Responsible AI (Capstone)

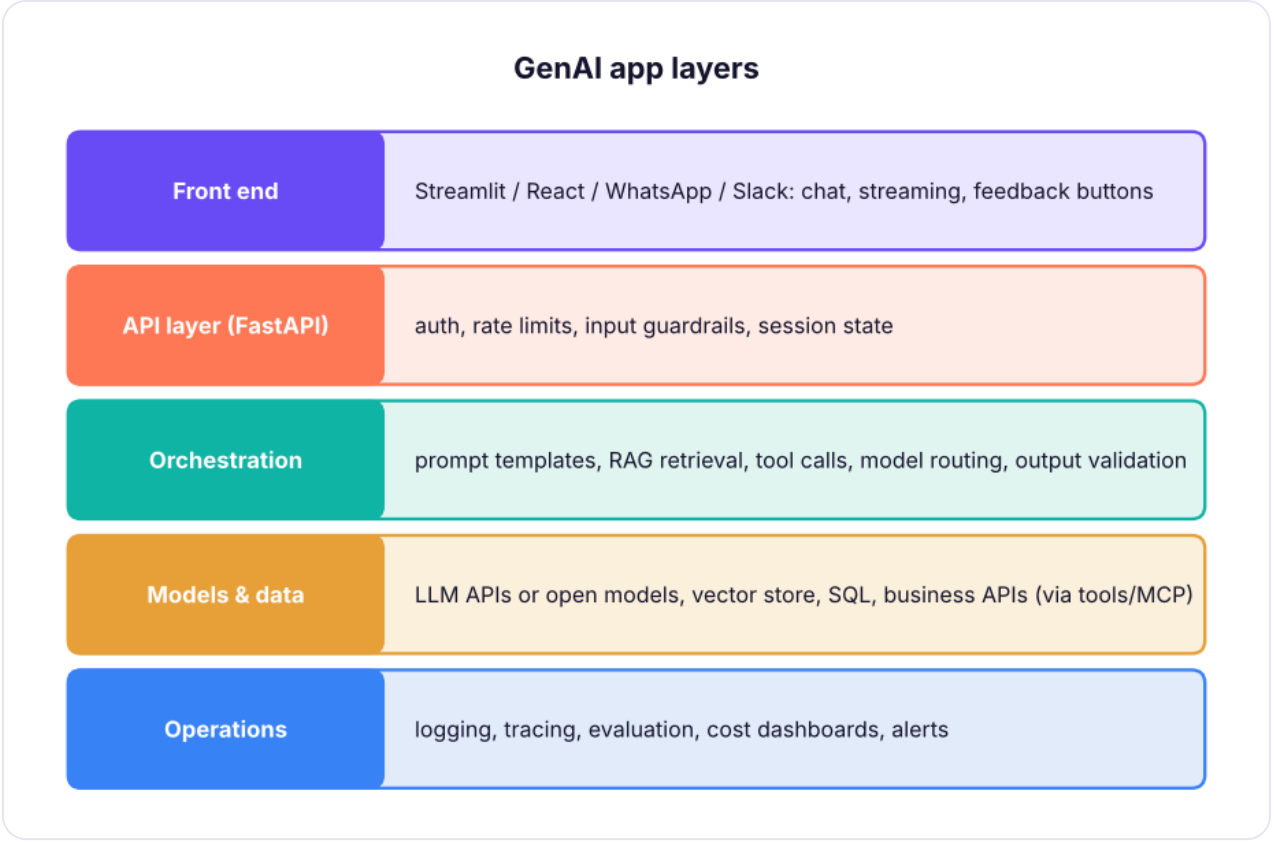
Put the GenAI track together. Reference architecture for an LLM application, a support-assistant capstone (RAG + tools + guardrails + evaluation), a Streamlit chat front end and a FastAPI back end, session memory, streaming, cost and latency budgets, caching, observability and tracing, deployment options, a launch checklist and responsible-AI governance, with a runnable cost and capacity planner.

GenAI Launchpad capstone · ✦ 2026 practice · Intermediate · ⏱ 150 min

By the end of this lesson you can

- Design the architecture of a production LLM application
- Build a chat UI and an API around an LLM pipeline
- Plan cost, latency and capacity with caching and model routing
- Add observability, feedback and evaluation loops
- Ship responsibly with a launch checklist and governance

Reference architecture



Components of a production GenAI application.

Capstone brief: NovaMart support assistant

Answer customer questions from the policy handbook (RAG, lesson 5), look up order status with a tool (lesson 6), refuse out-of-scope or unsafe requests, hand over to a human when unsure, and measure quality (lesson 9).

Requirement	Design
Accurate policy answers with sources	Hybrid retrieval + citations, "answer only from documents"
Order status	`get_order(order_id)` tool, read-only, customer must be authenticated
Refunds above ₹5,000	Tool creates a ticket for a human; the model never approves money
PII	Redact before logging; no card numbers in prompts
Quality	60-question eval set, weekly regression, thumbs up/down
Cost	Small model for routing/classification, larger model for complex answers, prompt caching

Chat front end (Streamlit)

Python

```python
# app.py  → streamlit run app.py
import streamlit as st, requests
st.title("NovaMart Help")
if "history" not in st.session_state: st.session_state.history = []
for m in st.session_state.history: st.chat_message(m["role"]).write(m["content"])
if q := st.chat_input("Ask about orders, returns, delivery…"):
    st.chat_message("user").write(q)
    r = requests.post("http://localhost:8000/chat", json={"session_id":
st.session_state.get("sid", "demo"), "message": q}, timeout=60).json()
    st.chat_message("assistant").write(r["answer"] + ("\n\n" + " · ".join(r["sources"]) if
r["sources"] else ""))
    st.session_state.history += [{"role": "user", "content": q}, {"role": "assistant",
"content": r["answer"]}]
    c1, c2 = st.columns(8)[:2]
    c1.button("👍", key=f"up{len(st.session_state.history)}"); c2.button("👎",
key=f"down{len(st.session_state.history)}")
```

API back end (FastAPI)

Python

```python
# api.py → uvicorn api:app
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import anthropic, time, logging
app, client = FastAPI(), anthropic.Anthropic()
SESSIONS: dict[str, list] = {}                          # use Redis or a database in
production
class ChatIn(BaseModel):
    session_id: str = Field(max_length=64)
    message: str = Field(min_length=1, max_length=2000)
@app.post("/chat")
def chat(body: ChatIn):
    if injection_score(body.message)[0] ≥ 2: raise HTTPException(400, "Please rephrase
your question.")
    prompt, ctx = build_prompt(body.message)            # RAG from lesson 5
    history = SESSIONS.setdefault(body.session_id, [])[-6:]          # keep the last 3
exchanges
    t0 = time.perf_counter()
    resp = client.messages.create(model="claude-sonnet-5-5", max_tokens=2000, system=
[{"type": "text", "text": SYSTEM, "cache_control": {"type": "ephemeral"}}],
                                  messages=history + [{"role": "user", "content": prompt}])
    answer = next(b.text for b in resp.content if b.type == "text")
    issues = check_output(answer)                         # lesson 9 guardrails
    if issues ≠ ["ok"]: answer = "Let me connect you with a support agent."
    SESSIONS[body.session_id] = history + [{"role": "user", "content": body.message},
{"role": "assistant", "content": answer}]
    logging.info({"latency_ms": int(1000 * (time.perf_counter() - t0)), "in":
resp.usage.input_tokens, "out": resp.usage.output_tokens, "issues": issues})
    return {"answer": answer, "sources": ctx.id.tolist()}
```

Cost and capacity planning

Python

```
import pandas as pd, numpy as np
PRICE = {"claude-haiku-5-5": (0.10, 0.50), "claude-sonnet-5-5": (2.00, 10.00), "claude-
opus-5-5": (4.00, 20.00)}   # USD per million tokens, Oct 2026 list prices
def monthly(conversations, turns, system_tok, ctx_tok, user_tok, out_tok, model,
cache_hit=0.0, usd_inr=88):
    pin, pout = PRICE[model]; calls = conversations * turns
    cached = system_tok * cache_hit; uncached = system_tok - cached + ctx_tok + user_tok
    usd = calls * ((uncached * pin + cached * pin * 0.1) + out_tok * pout) / 1e6
    return round(usd * usd_inr)
base = dict(conversations=60_000, turns=4, system_tok=1500, ctx_tok=1200, user_tok=60,
out_tok=180)
rows = []
for m in PRICE:
    for hit in (0.0, 0.9):
        rows.append({"model": m, "system prompt cache hit": f"{hit:.0%}", "₹ / month":
monthly(model=m, cache_hit=hit, **base)})
plan = pd.DataFrame(rows); print(plan.to_string(index=False))
route = 0.8 * monthly(model="claude-haiku-5-5", cache_hit=.9, **base) + 0.2 *
monthly(model="claude-sonnet-5-5", cache_hit=.9, **base)
print(f"\nrouting 80% of turns to Haiku and 20% to Sonnet: ≈ ₹{route:,.0f}/month for
{base['conversations']:,} conversations ({route / base['conversations']:.2f} ₹ per
conversation)")
```

OUTPUT

```
            model system prompt cache hit  ₹ / month
 claude-haiku-5-5                      0%       7730
 claude-haiku-5-5                     90%       5164
claude-sonnet-5-5                      0%     154598
claude-sonnet-5-5                     90%     103277
  claude-opus-5-5                      0%     309197
  claude-opus-5-5                     90%     206554

routing 80% of turns to Haiku and 20% to Sonnet: ≈ ₹24,787/month for 60,000
conversations (0.41 ₹ per conversation)
```

Levers: route simple turns to small models, cache the stable system prompt and documents, retrieve fewer but better chunks, cap output length, summarise long histories, and batch offline jobs.

Latency

Stage	Typical	Reduce by
Retrieval	20–200 ms	Index tuning, caching frequent queries
Model time to first token	0.3–2 s	Smaller model, shorter prompt, prompt caching
Generation	20–100 tokens/s	Shorter answers, streaming to the UI
Tool calls	API dependent	Parallel calls, timeouts

Stream responses so users see progress immediately; set timeouts and fallbacks for every external call.

Observability and feedback

Log every request with a trace: user/session (pseudonymised), prompt version, retrieved chunk ids, tool calls, model, tokens, latency, guardrail results, and user feedback. Tools: Langfuse, LangSmith, Arize Phoenix, Helicone, OpenTelemetry with your existing dashboards. Review low-rated conversations weekly, add them to the eval set, fix, and re-run the regression suite.

Deployment options

Option	Good for
Streamlit Community Cloud, Hugging Face Spaces	Demos and portfolios
Render, Railway, Fly.io, Cloud Run	Small production apps (containerised FastAPI)
AWS (Bedrock + Lambda/ECS), Azure (AI Foundry + App Service), GCP (Vertex AI + Cloud Run)	Enterprise, data residency, existing cloud
WhatsApp Business API, Slack, Teams	Meeting users where they already are

Launch checklist

1. Eval set passes target scores; regression suite in CI.
2. Guardrails: injection, PII, scope, output validation, refusal handling; human handover path.
3. Security: secrets in a vault, least-privilege tools, authentication, rate limits.
4. Privacy: DPDP Act notice and consent, retention limits, provider data settings.

5. Cost and latency budgets with alerts; kill switch and rollback to the previous prompt.
6. Monitoring dashboards and an on-call owner.
7. Documentation: model card / system card, known limitations, escalation contacts.
8. Gradual rollout (internal → 5% → 50% → 100%) with an A/B comparison against the old process.

Responsible AI governance

Be transparent that users are talking to AI; keep humans accountable for consequential decisions (refunds, credit, hiring); test for bias across languages and customer groups; follow emerging regulation (India's DPDP Act and IT rules, the EU AI Act for EU users, sector rules such as RBI guidelines); and keep an incident process for harmful outputs.

Summary

KEY TAKEAWAYS

- A GenAI app = front end + API + orchestration (prompts, RAG, tools, routing, validation) + models/data + operations.
- Streamlit gives a quick chat UI; FastAPI exposes a validated, authenticated, rate-limited backend.
- Plan cost per conversation; route easy turns to small models, cache stable prompts, cap outputs.
- Trace everything, collect feedback, and feed failures into the evaluation set.
- Launch with a checklist: evals, guardrails, security, privacy, budgets, monitoring, gradual rollout and human accountability.

Quick quiz

6 questions · tap an answer to check it

1. Where should secrets such as API keys live in a deployed app?

A In a secrets manager or environment variables, never in code

B In the front end JavaScript

C In the prompt

D In the README

Answer: A. Rotate if exposed.

2. Which lever usually cuts cost most without hurting quality on simple turns?

A Routing simple requests to a small model

B Raising max_tokens

C Disabling caching

D Adding more chunks

Answer: A. Use big models only where needed.

3. Why stream responses?

A Users see text immediately and long answers avoid timeouts

B It lowers token cost

C It improves accuracy

D It is required

Answer: A. Better perceived latency.

4. A refund over ₹5,000 is requested. What should the assistant do?

- **A** Create a ticket for a human; never approve money itself
- **B** Approve it
- **C** Ignore it
- **D** Ask for card details in chat

Answer: A. Keep humans accountable.

5. What should happen to conversations users rate 👎?

- **A** Review them, add failures to the eval set, fix and re-run regressions
- **B** Delete them
- **C** Nothing
- **D** Retrain the base model immediately

Answer: A. Close the feedback loop.

6. What is a safe rollout strategy?

- **A** Gradual exposure with monitoring and a rollback switch
- **B** 100% launch on day one
- **C** No monitoring
- **D** Launch without evaluation

Answer: A. Reduce blast radius.

EXERCISE

Capstone tasks

(1) Combine the RAG pipeline (lesson 5), the order tool (lesson 6) and the guardrails (lesson 9) into one `answer(question, user)` function. (2) Build the Streamlit UI and FastAPI backend locally. (3) Write a 30-question eval set and report scores. (4) Estimate the monthly cost for your expected traffic and write a one-page launch plan.

Show solution (worked answer)

Use the code in lessons 5, 6 and 9 as building blocks; keep each step testable. Report: eval scores by question type, average latency, cost per conversation, open risks and the rollout plan.

INTERVIEW CORNER

Questions you may be asked

Q: How would you take an LLM prototype to production? Add evaluation and regression tests, guardrails, authentication and rate limits, observability, cost and latency budgets, privacy compliance, gradual rollout with monitoring, and clear human escalation.

Q: How do you control the cost of a chatbot? Model routing, prompt caching, concise retrieval, output limits, summarised history, batching offline work and dashboards with alerts on cost per conversation.

Q: What would you monitor after launch? Quality (feedback, judge scores), safety (guardrail hits, refusals), cost, latency, errors and drift in user questions.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.